Prototype Speech Recognizer

Advanced Topics in Speech Processing

CS60116

by

Aarobh Kumar

Roll Number: 25CS60R56

Apeksha Subhash Gulhane

Roll Number: 25CS60R58

Under the Supervision of

Prof. K. Sreenivasa Rao

Department of Computer Science and Engineering

Indian Institute of Technology Kharagpur

April 2026

Contents

Chapter 1

Abstract

This report presents the complete design and implementation of a Prototype Speech Recognizer developed for the course *Advanced Topics in Speech Processing* (CS60116) at IIT Kharagpur, under the supervision of Prof. K. S. Rao. The project evolved through four progressive phases, each exploring a different point in the spectrum from classical feature-engineering to modern pre-trained deep learning.

The **first phase** implemented a word-level classification pipeline using the Google Speech Commands dataset (35 classes, $\sim$105,000 samples). Features were extracted as 39-dimensional MFCC vectors (13 coefficients with first- and second-order delta coefficients) and classified with ten scikit-learn models including SVM, Random Forest, XGBoost, MLP, and KNN. A CNN-based deep learning model was subsequently trained on 2-D MFCC feature maps (shape $40 \times 44 \times 1$) and validated through a live microphone prediction demo.

The **second phase** built a sentence-level ML pipeline using a 100-sentence TTS corpus augmented with Gaussian noise, time-stretching, and pitch-shifting. Four classifiers were evaluated on 120-dimensional MFCC features, with SVM achieving 88–95% accuracy.

The **third phase** scaled to open-vocabulary continuous speech recognition using a real large-scale corpus. A Conv1D + Bidirectional LSTM architecture trained with Connectionist Temporal Classification (CTC) loss was applied to the LibriSpeech *train-clean-100* dataset ($\sim$28,000 utterances). The custom model achieved a Word Error Rate of 0.77 and a Character Error Rate of approximately 0.50, compared to 0.31 WER and 0.10 CER for the Whisper `base` model used as a benchmark.

The **fourth phase** developed a browser-based real-time transcription application powered by OpenAI Whisper via a Python FastAPI backend, supporting live spectrogram visualization, three model sizes, noise augmentation, Vosk offline comparison, and Word Error Rate evaluation. The Whisper-based system achieved near-zero WER on clean English speech.

Chapter 2

Introduction

2.1 What is Speech Recognition?

Automatic Speech Recognition (ASR) is the technology that enables a computer to interpret spoken language and convert it into text or commands. When a person speaks, the vocal tract produces pressure waves that a microphone converts into a digital signal, typically sampled at 16 kHz. The central challenge of ASR is to extract meaningful linguistic content from this continuous acoustic signal while remaining robust to variation in speaker identity, speaking rate, accent, and background noise.

Early ASR systems in the 1970s and 1980s could only handle isolated words from small vocabularies. Hidden Markov Models (HMMs) introduced in the 1980s enabled continuous speech recognition and remained the dominant framework for nearly two decades. The widespread adoption of deep learning in the 2010s dramatically reduced error rates, and transformer-based models such as Whisper now approach human-level performance on several standard benchmarks.

2.2 Importance and Applications

Speech is the most natural form of human communication. ASR technology underpins virtual assistants (Alexa, Siri, Google Assistant), meeting transcription services, accessibility devices for people with disabilities, voice search, call-centre analytics, automotive hands-free control, and smart home automation. Building a working prototype ASR system integrates signal processing, machine learning, and software engineering in a single pipeline, making it an ideal capstone exercise for a course in speech processing.

2.3 Project Motivation

This project was designed as a structured progression through the history and current state of ASR. Rather than adopting a single method, the team moved deliberately from classical feature-engineering (MFCC + scikit-learn classifiers), through deep convolutional learning on real word-level data, to end-to-end sequence modeling with CTC on a large continuous-speech corpus, and finally to a modern pre-trained transformer. This progression allows the strengths and limitations of each paradigm to be observed empirically and compared within a single coherent project.

Chapter 3

Literature Review

The field of ASR has evolved through several distinct paradigms, each of which directly shaped the methodology adopted in this project.

Davis and Mermelstein [**?**] introduced Mel Frequency Cepstral Coefficients (MFCCs) as a compact acoustic representation aligned with the frequency resolution of human auditory perception. Furui [2] demonstrated that appending first-order delta coefficients—temporal derivatives of the cepstral sequence—captures spectral dynamics and consistently improves recognition. Both techniques form the backbone of feature extraction in Phases 1 and 2 of this project.

Rabiner [3] established HMMs as the dominant ASR framework, modeling speech as a sequence of hidden phoneme states with Gaussian Mixture Model observation densities. HMM-GMM systems remained the practical state of the art for two decades. Hinton et al. [4] showed that replacing the GMM component with a deep neural network substantially reduced word error rates, initiating the modern deep learning era in ASR.

Graves et al. [5] introduced Connectionist Temporal Classification (CTC), a loss function that enables fully end-to-end training of sequence-to-sequence models without requiring frame-level alignment between audio and transcript. CTC is the training objective used in Phase 3 of this project. The transformer architecture introduced by Vaswani et al. [6] and attention-based encoder-decoder models such as Listen, Attend and Spell [7] subsequently completed the shift from hybrid HMM-DNN systems to purely end-to-end sequence models.

Radford et al. [8] introduced Whisper, a transformer encoder-decoder trained on 680,000 hours of weakly supervised multilingual audio using a multi-task formulation. Whisper achieves near state-of-the-art accuracy without task-specific fine-tuning and is used as both a benchmark and the production backend in Phase 4, via `faster-whisper` with int8 CPU quantization. For data augmentation, Ko et al. [9] showed that speed perturbation is a simple yet effective technique for increasing acoustic diversity, while Park et al. [10] introduced SpecAugment, which applies frequency and time masking directly to the spectrogram. Warden [11] released the Speech Commands dataset used in Phases 1 and 2, and Panayotov et al. [12] released LibriSpeech, used in Phase 3. Word Error Rate (WER), computed from Levenshtein edit distance at the word level, is the primary evaluation metric throughout.

Chapter 4

System Overview

The project is organized into four subsystems developed sequentially, with each phase building on the conceptual and practical lessons of the previous one.

4.1 Phase 1 — Word-Level Classification (Notebooks 01–06)

The first subsystem implements a complete machine learning and deep learning pipeline for recognizing 35 spoken command words from the Google Speech Commands dataset. The work is structured as six Jupyter notebooks covering, in order: dataset exploration and visualization, MFCC feature extraction for classical models, training and evaluation of ten scikit-learn classifiers, 2-D MFCC feature extraction for the CNN, CNN training and evaluation, and a live microphone prediction demo.

4.2 Phase 2 — Sentence-Level ML Pipeline

The second subsystem extends recognition from individual words to complete command sentences. A 100-sentence TTS corpus is constructed, augmented to increase acoustic diversity, and used to train and compare four scikit-learn classifiers on 120-dimensional MFCC features. This phase demonstrates that the MFCC-plus-classifier paradigm can generalize from single tokens to short phrases.

4.3 Phase 3 — End-to-End Deep Learning on LibriSpeech

The third subsystem scales continuous speech recognition to a large real-speech corpus. A Conv1D + Bidirectional LSTM model is trained with CTC loss on the LibriSpeech *train-clean-100* set and evaluated on *test-clean*. This phase directly demonstrates the challenge of open-vocabulary ASR when training from scratch and quantifies the performance gap relative to a large pre-trained model.

4.4 Phase 4 — Real-Time Web Application

The fourth subsystem is a browser-based application with a Python FastAPI backend running Whisper ASR. The user opens the web interface, clicks Record, and speaks into the microphone. The browser captures audio via the MediaRecorder API, sends it to the backend, and receives the recognized transcript as JSON. The frontend renders the transcript alongside live waveform and spectrogram visualizations and, optionally, a Word Error Rate evaluation against a reference.

4.5 Shared Architecture

All four phases share a common three-layer structure. The **Input Layer** accepts audio from the microphone, uploaded WAV or MP3 files, or TTS-generated clips, all standardized to 16 kHz mono. The **Processing Layer** either extracts MFCC features and applies a trained classifier (Phases 1–2), feeds time-series MFCC frames through a CTC-trained neural model (Phase 3), or passes audio directly to the Whisper backend (Phase 4). The **Output Layer** renders the recognized text, accuracy metrics or WER, and visualizations such as waveform, spectrogram, and MFCC heatmap.

Chapter 5

Datasets

5.1 Google Speech Commands v0.02 (Phases 1 and 2)

Table 5.1: Speech Commands Dataset Summary

Property	**Value**
Total audio files	∼105,000
Audio format	16-bit PCM WAV
Clip duration	∼1 second per clip
Sample rate	16,000 Hz
Language	English
Utterance type	Single spoken words
Number of classes used	35

The dataset includes 20 core command words (*yes, no, up, down, left, right, on, off, stop, go, zero, one, two, three, four, five, six, seven, eight, nine*) and 15 auxiliary words (*bed, bird, cat, dog, happy, house, marvin, sheila, tree, wow, backward, forward, follow, learn, visual*). A `_background_noise_` directory was excluded from classification. Train, validation, and test splits are determined using a hash-based file assignment method to ensure that utterances from the same speaker do not appear in more than one split. The dataset was downloaded from:

`http://download.tensorflow.org/data/speech_commands_v0.02.tar.gz`

5.2 Synthetic TTS Corpus (Phase 2)

Training data for Phase 2 was generated synthetically using the `espeak-ng` text-to-speech engine, avoiding the cost and time of recording real speech. The vocabulary covers 100 English command sentences distributed across ten semantic domains: greetings, home automation, device control, navigation, reminders, media queries, scheduling, entertainment, weather, and general information questions. Five virtual speakers at different speaking rates (120, 135, 150, 165, and 180 words per minute) each synthesized all 100 sentences, producing a base corpus of 500 WAV files sampled at 16 kHz. After applying five augmentation strategies, the final dataset contains 1,200 training samples.

5.3 LibriSpeech (Phase 3)

LibriSpeech is a large-scale read-speech corpus derived from LibriVox audiobooks, released by Panayotov et al. [12]. Phase 3 uses two subsets.

Table 5.2: LibriSpeech Subsets Used in Phase 3

Subset	**Approx. Hours**	**Purpose**
train-clean-100	100 h	Model training
test-clean	5 h	Evaluation

Audio is provided as 16 kHz FLAC files organized by speaker and chapter identifiers. Ground-truth transcripts are stored in `.trans.txt` files with one line per utterance, each prefixed with the utterance identifier. The preprocessing pipeline walks the directory tree and pairs each audio file with its transcript by matching these identifiers.

5.4 Real Microphone and Uploaded Audio (Phase 4)

The Whisper-based web application in Phase 4 operates on real microphone recordings captured in the browser and on uploaded WAV or MP3 files. This provides a direct, user-facing evaluation of the Whisper pre-trained model on naturalistic speech, as opposed to the controlled or synthetic conditions of the earlier phases.

Chapter 6

Methodology

6.1 Phase 1 — MFCC Feature Extraction for Classical Models (Notebook 02)

Each 1-second audio clip from the Speech Commands dataset was loaded at 16 kHz. Thirteen MFCC coefficients were computed from the raw waveform. First-order delta coefficients and second-order delta-delta coefficients were then derived using a window width of 3 frames. The three coefficient matrices were stacked vertically to form a $39 \times T$ representation, which was averaged across the time axis to produce a single 39-dimensional feature vector per clip. This procedure yielded a feature matrix of shape $(105{,}829 \times 39)$, with one row per audio file.

6.2 Phase 1 — 2-D MFCC Feature Extraction for CNN (Notebook 04)

For the convolutional model, a 2-D MFCC image was extracted from each clip rather than a collapsed vector. Forty MFCC coefficients were computed, and the time axis was padded with zeros or truncated to exactly 44 frames, yielding a fixed-size 40×44 matrix. A single channel dimension was appended to obtain the shape $(40, 44, 1)$, treating the feature map as a grayscale image. Two optional data augmentation functions were defined: `add_noise` adds Gaussian noise scaled by a factor of 0.005, and `shift_audio` applies a random circular time shift of up to 1,600 samples. The complete CNN feature dataset had shape $(105{,}829,\ 40,\ 44,\ 1)$.

6.3 Phase 1 — Classical ML Training (Notebook 03)

The 39-dimensional feature matrix was split 80/20 into training and test sets with `random_state=42`. A `StandardScaler` was applied to normalize all features to zero mean and unit variance before each classifier. Ten models were trained and evaluated, as listed in Table 6.1.

Table 6.1: Classical Classifiers Evaluated on 39-Dimensional MFCC Features

Model	**Key Hyperparameters**	**Notes**
Logistic Regression	`max_iter=1000`	Linear baseline
SVM (RBF Kernel)	Default `C`, `gamma`	Strong in high-dim spaces
Linear SVM	`LinearSVC`	Faster than RBF SVM
SGD Classifier	`loss=hinge`	Scalable SVM approximation
Random Forest	`n_estimators=100`	Ensemble; 35-class native support
Gradient Boosting	Default settings	Sequential ensemble
Hist Gradient Boosting	Default settings	Efficient tree boosting
MLP	`hidden=(256,128)`, `max_iter=20`	Neural baseline
XGBoost	`n_estimators=200`, `lr=0.1`	Best tree-based model
KNN	`n_neighbors=5`	Distance-based baseline

Labels were encoded with scikit-learn's `LabelEncoder`. SVM (RBF) and XGBoost consistently achieved the highest accuracy on the 35-class task.

6.4 Phase 1 — CNN Training (Notebook 05)

A sequential CNN was constructed in TensorFlow/Keras. Three convolutional blocks, each comprising a Conv2D layer, Batch Normalization, and MaxPooling2D, extracted hierarchical spatial features from the 2-D MFCC image. Dropout layers with a rate of 0.3 were inserted after the third pooling operation and after the first fully connected layer to reduce overfitting. The architecture is detailed in Table 6.2.

Table 6.2: CNN Architecture for Word-Level Speech Recognition

Layer	**Filters / Units**	**Kernel**	**Notes**
Conv2D	32	3×3	ReLU activation
BatchNormalization	—	—	
MaxPooling2D	—	2×2	
Conv2D	64	3×3	ReLU activation
BatchNormalization	—	—	
MaxPooling2D	—	2×2	
Conv2D	128	3×3	ReLU activation
BatchNormalization	—	—	
MaxPooling2D	—	2×2	
Dropout	—	—	rate = 0.3
Flatten	—	—	
Dense	256	—	ReLU activation
Dropout	—	—	rate = 0.3
Dense (output)	35	—	Softmax activation

The model was compiled with the Adam optimizer and sparse categorical cross-entropy loss, then trained for 10 epochs with a batch size of 128. Input features were normalized by dividing by 255. The trained model was saved as `cnn_speech_model.keras`.

6.5 Phase 1 — Live Prediction Demo (Notebook 06)

A complete end-to-end prediction function was implemented to validate the CNN on real spoken input. The pipeline records 1 second of audio at 16 kHz using the `sounddevice` library, extracts the 40×44 MFCC map, appends batch and channel dimensions to obtain shape $(1, 40, 44, 1)$, runs the CNN, and maps the `argmax` of the 35-way softmax output to the corresponding word label. Batch testing across the words *yes, no, up, down* confirmed correct predictions in all four cases, and a real microphone recording was also classified correctly.

6.6 Phase 2 — Sentence-Level ML Pipeline

6.6.1 Corpus Construction and Augmentation

Five augmentation strategies were applied cyclically to the 500 base TTS clips to produce a final training set of 1,200 samples. The strategies were: (1) **Clean** — raw TTS output used without modification; (2) **Noisy** — Gaussian white noise added at a random SNR uniformly drawn from 10 to 20 dB; (3) **Time-Stretched** — utterance duration scaled between $0.85\times$ and $1.15\times$ without altering pitch; (4) **Pitch-Shifted** — fundamental frequency shifted between -3 and $+3$ semitones; and (5) **Noisy + Pitch-Shifted** — both noise injection and pitch perturbation applied simultaneously to simulate a different speaker in a noisy environment. All augmentation was performed offline using `librosa` before feature extraction.

6.6.2 120-Dimensional MFCC Feature Extraction

A fixed-length 120-dimensional feature vector was constructed from each augmented clip. A Short-Time Fourier Transform with 2,048-sample frames and a 512-sample hop was applied. The power spectrum was mapped through 40 Mel-scale triangular filters, converted to the log domain, and then passed through a Discrete Cosine Transform to yield 40 MFCC coefficients per frame. First-order delta coefficients were also computed. The final feature vector concatenated the frame-averaged mean of the 40 MFCCs (40 values), the frame standard deviation of the 40 MFCCs (40 values), and the frame-averaged mean of the 40 delta coefficients (40 values), giving $40 + 40 + 40 = 120$ features per utterance.

6.6.3 Classifier Evaluation

Four classifiers were trained inside scikit-learn `Pipeline` objects that apply `StandardScaler` normalization as the first step, as summarized in Table 6.3. The best-performing pipeline and the fitted `LabelEncoder` were serialized using `joblib`. A JSON metadata file recorded

the model name, accuracy, vocabulary list, sample rate, and MFCC configuration to support reproducibility and inference reuse.

Table 6.3: Sentence-Level ML Model Performance on 120-Dimensional MFCC Features

Model	**Test Accuracy**	**Training Time**	**Remarks**
SVM (RBF, C=10)	88–95%	15–25 s	Best overall performer
Random Forest (300 trees)	82–90%	20–35 s	Robust; low overfitting
Gradient Boosting (150 trees)	78–87%	60–120 s	Competitive but slow
KNN (k=5)	70–80%	$<$1 s	Weakest; limited by distance

6.7 Phase 3 — End-to-End ASR on LibriSpeech (Conv1D + BiLSTM + CTC)

6.7.1 Data Loading and Preprocessing

The LibriSpeech corpus was parsed by walking the directory trees of *train-clean-100* and *test-clean*. Each `.trans.txt` file was read line-by-line to pair each audio path with its ground-truth transcript. Transcripts were lowercased and filtered to retain only the 26 Latin letters and the space character, defining a vocabulary Σ of 27 symbols. Each audio file was loaded at 16 kHz using `librosa` and converted to a time-series of 40 MFCC coefficients per frame.

Samples were filtered by three criteria: the label must contain at least 5 characters, at most 200 characters for training (120 for testing), and the label length must be strictly less than the input sequence length (a necessary condition for CTC). Sequences were padded with zeros to the length of the longest training utterance; label sequences were padded with -1 as a sentinel value. After filtering, the training set contained approximately 28,000 utterances.

6.7.2 Model Architecture

The acoustic model is a sequence-to-sequence architecture processing the full MFCC frame sequence, which preserves all temporal ordering information — a fundamental improvement over the time-averaged feature vector used in Phases 1 and 2. Table 6.4 summarizes the network.

Table 6.4: Conv1D + BiLSTM CTC Model Architecture

Layer	**Units / Filters**	**Notes**
Input	$T \times 40$	Variable-length MFCC frame sequence
Conv1D	128 filters, kernel 3	ReLU; extracts local spectral patterns
BatchNormalization	—	
Bidirectional LSTM	256 units	Returns full output sequence
Dropout	—	rate = 0.3
Bidirectional LSTM	256 units	Returns full output sequence
Dense (output)	$\|\Sigma\| + 1 = 28$	Softmax over characters + CTC blank

The Conv1D layer extracts local spectral patterns across adjacent time frames. The two Bidirectional LSTM layers then model long-range temporal dependencies in both the forward and backward directions, producing a contextualized representation at each time step. The output dense layer produces a probability distribution over the 27 character symbols plus the CTC blank token at every frame.

6.7.3 CTC Training

The CTC loss function marginalizes over all valid alignments between the output probability sequence and the target label sequence, removing the need for frame-level annotation. A custom `ctc_loss` function computed input lengths as the full padded sequence length and derived label lengths by counting non-sentinel ($\neq -1$) positions in each padded label. The model was compiled with the Adam optimizer at a learning rate of 3×10^{-4} and trained with an `EarlyStopping` callback (patience = 5, monitoring training loss, restoring best weights) for up to 10 epochs with batch size 16 on a Google Colab GPU.

6.7.4 CTC Decoding

Character sequences were decoded from the output probability matrix using `tf.keras.backend.ctc_d` with beam search (beam width = 10). Decoded token indices were mapped back to characters via the `idx_to_char` dictionary, with the CTC blank token discarded.

6.8 Phase 4 — Whisper Web Application

6.8.1 System Flow

The end-to-end flow of the web application is as follows. The user clicks Record in the browser, which initiates microphone capture via the `MediaRecorder` API. Simultaneously, the Web Audio API `AnalyserNode` feeds a 60 fps rendering loop that draws a live time-domain waveform and a scrolling frequency-domain spectrogram. When the user clicks Stop, the accumulated audio blob is POSTed to the `/transcribe` endpoint on the local FastAPI server. The server loads the selected Whisper model, runs transcription with timing,

and returns the transcript and processing time as JSON. The frontend displays the result and, if a reference transcript is provided, computes and shows the Word Error Rate.

6.8.2 Whisper Model Sizes

Whisper is a transformer encoder-decoder that takes log-Mel spectrograms as 30-second chunks and generates text autoregressively. Three model sizes are exposed through the interface (Table 6.5). All models run on CPU via `faster-whisper` with int8 quantization, making the application usable on a standard laptop without a GPU.

Table 6.5: Whisper Model Sizes Available in the Web Application

Model	**Parameters**	**Characteristic**
`tiny`	39 M	Fastest; lower accuracy
`base`	74 M	Balanced speed and accuracy
`small`	244 M	Most accurate; slowest

Chapter 7

Implementation Details

7.1 Tools and Libraries

Table 7.1 lists the key technologies used across all four phases.

Table 7.1: Technologies Used Across All Phases

Technology	Purpose
Python 3.10+	Primary programming language
librosa	Audio loading, MFCC extraction, augmentation
scikit-learn	ML models, preprocessing, evaluation
TensorFlow / Keras	CNN (Phase 1) and BiLSTM-CTC (Phase 3)
XGBoost	Gradient boosted tree classifier
pyttsx3 / espeak-ng	TTS speech synthesis (Phase 2)
NumPy, pandas	Numerical data handling
matplotlib, seaborn	Visualization and evaluation plots
sounddevice	Microphone recording for live demo
pydub	Audio format conversion (MP3 to WAV)
FastAPI	Python HTTP backend (Phase 4)
faster-whisper	Efficient int8 Whisper inference
Vosk	Offline ASR baseline comparison
JavaScript Web Audio API	Browser waveform and spectrogram
MediaRecorder API	Browser microphone capture

7.2 Phase 1 Code Modules

Notebook 01 (Dataset Exploration) loads a sample WAV file and renders the time-domain waveform, the STFT-based log spectrogram, and a 13-coefficient MFCC heatmap using `librosa.display`. It confirms the dataset structure of 35 class directories containing $\sim$105,000 files in total.

Notebook 02 (MFCC Feature Extraction) iterates over all 35 class folders, extracts a 39-dimensional MFCC+delta+delta2 feature vector for each clip, encodes class labels using `LabelEncoder`, and saves `X_features.npy` and `y_labels.npy` for downstream use.

Notebook 03 (Classical Model Training) loads the saved features, applies `StandardScaler`,

trains all ten classifiers, and prints accuracy scores, full classification reports, and a confusion matrix for the best model.

Notebook 04 (CNN Feature Extraction) extracts fixed-size 40×44 MFCC images with zero-padding or truncation, appends the channel dimension, and saves `X_cnn_features.npy` and `y_cnn_labels.npy`.

Notebook 05 (CNN Training) loads the CNN features, normalizes by 255, applies a stratified 80/20 split, trains the three-block CNN for 10 epochs, evaluates test accuracy, saves the model, and plots both training accuracy curves and a confusion matrix heatmap.

Notebook 06 (Prediction Demo) loads the saved CNN and validates it on random WAV files from four word classes and on a real microphone recording captured with `sounddevice`.

7.3 Phase 3 Code Module

ATSP_Speech_Recognizer.ipynb implements the full LibriSpeech pipeline on Google Colab. The corpus is mounted from Google Drive. A `load_librispeech` function parses the directory tree and matches each audio file to its transcript. A `prepare_data` function extracts MFCC sequences, filters samples by length constraints, and encodes characters. Sequences are padded using `pad_sequences`. The Conv1D + BiLSTM model is built in Keras with a custom CTC loss function and trained using early stopping. The trained model is saved to Google Drive. A `decode_prediction` function applies CTC beam search (width = 10) and maps decoded indices back to characters.

7.4 Phase 4 Web Application Modules

server.py is the FastAPI backend. The `/transcribe` endpoint accepts a multipart audio file upload, loads the selected Whisper model, runs transcription with wall-clock timing, and returns the transcript and processing time as JSON. The `/analyze-audio` endpoint decodes the uploaded audio and computes a compact spectrogram, MFCC feature map, RMS energy level, estimated noise floor, and estimated SNR, returning all visualizations as base64-encoded PNG images.

app.js handles all browser-side logic and real-time canvas rendering. The `MediaRecorder` API captures microphone audio; the `AnalyserNode` drives 60 fps waveform and spectrogram rendering. On stop, the audio blob is posted to `/transcribe` and the returned transcript is displayed. Word Error Rate evaluation, model comparison mode, batch file upload, Vosk offline baseline, and noise augmentation are all implemented as independent asynchronous JavaScript functions.

index.html / styles.css constitute a fully self-contained single-page application featuring a recording panel, transcript area, model selector (tiny/base/small), language selector, waveform and spectrogram canvases, WER evaluator, speaker and environment metadata fields, dataset upload section, and bar charts of WER and processing time by model.

Chapter 8

Results and Analysis

8.1 Phase 1 — Word-Level Classification

8.1.1 Classical ML on 39-Dimensional MFCC Features

All ten classifiers were evaluated on a 20% held-out test set drawn from 105,829 samples across 35 classes. SVM (RBF kernel) and XGBoost consistently achieved the highest accuracy on this task. The RBF-SVM exploits its ability to construct non-linear decision boundaries in the 39-dimensional feature space, while XGBoost benefits from its boosted ensemble of shallow trees. Logistic Regression served as a strong linear baseline, outperforming KNN despite its greater simplicity. The MLP underperformed its theoretical capacity due to the severe constraint of only 20 training iterations on a large, 35-class dataset. KNN was the weakest model overall, indicating that the MFCC feature space is not sufficiently locally clustered for distance-based classification at this scale.

8.1.2 CNN on 2-D MFCC Feature Maps

The CNN trained for 10 epochs with batch size 128. The three-block convolutional architecture with Batch Normalization and Dropout achieved strong generalization on the 35-class task. Training and validation accuracy converged smoothly, indicating the absence of significant overfitting. The live demo in Notebook 06 confirmed correct predictions for random samples of *yes, no, up, down*, and a microphone recording spoken during testing was also classified correctly.

8.2 Phase 2 — Sentence-Level Results

SVM achieved the highest accuracy on the 100-sentence TTS corpus, as expected for a high- dimensional MFCC feature space where inter-class boundaries in a fixed vocabulary are well-defined. KNN was the weakest performer, reflecting the difficulty of nearest-neighbor classification when only 9–12 training examples are available per class in a 120-dimensional space. During informal testing on 100 random training samples, the SVM correctly identified most phrases with confidence above 80% for clean and mildly noisy conditions. A representative inference output is shown below.

```
Predicted: "turn on the lights"  (Confidence: 87.4%)
Top-3: turn on the lights  (87.4%)
       switch off all lights  ( 6.2%)
       dim the lights          ( 3.1%)
```

However, when the model was informally evaluated on naturally spoken speech (rather than TTS audio), accuracy dropped substantially, confirming the domain mismatch problem inherent to TTS-based training.

8.3 Phase 3 — LibriSpeech Conv1D + BiLSTM + CTC

The custom acoustic model was evaluated on the LibriSpeech *test-clean* split. Results are compared against the Whisper `base` model used as a pre-trained benchmark (Table 8.1).

Table 8.1: WER and CER Comparison on LibriSpeech *test-clean*

Model	**WER**	**CER**	**Notes**
Conv1D + BiLSTM + CTC (ours)	0.77	≈0.50	Trained from scratch, 10 epochs
Whisper `base`	0.31	≈0.10	Pre-trained, no fine-tuning

The custom model's WER of 0.77 reflects the inherent difficulty of open-vocabulary continuous speech recognition when training from scratch on only 100 hours of data and 10 training epochs. The model successfully learned to produce approximately correct character sequences for short, clearly articulated utterances, but struggled with longer sentences, phonetically similar words, and reduced forms of function words. The large performance gap relative to Whisper directly illustrates the value of pre-training at scale: Whisper's exposure to 680,000 hours of multilingual audio gives it acoustic and linguistic knowledge that no 100-hour corpus can replicate.

8.4 Phase 4 — Whisper Web Application

The web application was tested on short English utterances recorded through a laptop microphone in a quiet room (Table 8.2).

Table 8.2: Whisper Model Comparison on Clean-Room English Utterances

Model	**WER (%)**	**Avg. Time (s)**	**Parameters**
`tiny`	3–8	0.5–1.0	39 M
`base`	1–4	1.0–2.5	74 M
`small`	0–2	3.0–6.0	244 M

The `small` model produced the lowest WER, frequently yielding exact transcripts (WER = 0%) for short command utterances. At 10 dB SNR with added synthetic white noise, WER rose to 8–15% for `tiny` and 3–7% for `base`, demonstrating that Whisper is robust under moderate noise but not immune to severe degradation.

8.5 Word Error Rate Analysis

The WER module used in Phases 3 and 4 decomposes recognition errors into three categories (Table 8.3). The formula is:

$$\text{WER} = \frac{S + D + I}{N} \tag{8.1}$$

where S is substitutions, D is deletions, I is insertions, and N is the number of words in the reference transcript. Errors are computed from the word-level Levenshtein alignment between the hypothesis and the reference.

Table 8.3: Word Error Rate Error Categories

Error Type	**Meaning**	**Example**
Substitution	Reference word replaced by a different word	*browser* $\rightarrow$ *closer*
Deletion	Reference word absent from the hypothesis	*check* not transcribed
Insertion	Extra word present in the hypothesis	spurious *the*

Chapter 9

Challenges Faced

Dataset scale and extraction time. The Speech Commands dataset contains over 105,000 audio files. MFCC extraction in Notebooks 02 and 04 required approximately 7 and 10 minutes respectively on a local CPU, making early experimentation slow and discouraging rapid iteration.

Fixed-length feature aggregation. Averaging the MFCC sequence into a single mean vector discards all temporal ordering information. Utterances with similar phoneme distributions but different word order can produce nearly identical 39-dimensional vectors, which is a fundamental limitation of the approach for any vocabulary larger than isolated words.

Synthetic vs. natural speech mismatch. The Phase 2 sentence classifier was trained entirely on `espeak-ng` TTS audio, which is acoustically very different from natural human speech. When informally evaluated on real voice recordings, recognition accuracy dropped substantially. Overcoming this mismatch requires either real recorded training data or fine-tuning a pre-trained model.

CTC training instability. Training the BiLSTM-CTC model from scratch on 100 hours of read speech is a challenging optimization problem. The loss exhibited instability in early epochs, requiring careful tuning of the Adam learning rate (3×10^{-4}) and early stopping patience to achieve consistent convergence.

Platform differences for espeak-ng. The TTS engine generates slightly different acoustic output on different operating systems. Achieving consistent output between the local Windows development environment and the Google Colab/Kaggle Linux runtime required debugging the audio generation parameters.

Whisper model download time. The first use of any Whisper model requires downloading pre-trained weights from the internet. The `small` model is approximately 500 MB, which is inconvenient during time-constrained live demonstrations.

Browser microphone over HTTP. Modern browsers enforce the HTTPS requirement for microphone access through the MediaRecorder API. Running the application locally over plain HTTP required enabling experimental browser flags, which is unsuitable for any production or shared deployment.

Limited samples per class in Phase 2. With 1,200 total samples and 100 sentence classes, each class has approximately 9–12 training examples on average, which is insufficient for reliable classification in a 120-dimensional feature space. Expanding the corpus to at least 50 examples per class would substantially improve all four models.

Chapter 10

Future Work

Real speech data. Replacing the TTS corpus with real recorded speech from multiple speakers would dramatically reduce the domain mismatch observed in Phase 2. Mozilla Common Voice or the FLEURS corpus provide freely available multilingual real-speech data suitable for this purpose.

Improved sequence models. A larger BiLSTM, a Conformer block, or training on more data would reduce the WER of the custom end-to-end model from Phase 3. Shallow fusion of a character-level language model with the CTC decoder is a practical and well-understood improvement that requires no change to the acoustic model architecture.

Fine-tuning Whisper. Fine-tuning the Whisper `small` model on a specific command vocabulary using the Hugging Face `transformers` library would improve accuracy for domain-specific terminology while preserving the general robustness conferred by large-scale pre-training.

Streaming recognition. The current web application transcribes only after recording stops. A WebSocket-based streaming architecture would push incremental transcript updates to the browser as the user speaks, dramatically improving the interactive experience.

Noise suppression. Integrating a lightweight front-end noise suppressor such as RNNoise before passing audio to the ASR model would improve performance under real-world conditions beyond what offline noise augmentation can simulate.

Indian language support. Whisper natively supports Hindi and several other Indian languages. Extending the web application to these languages and fine-tuning on corpora such as IIT Madras SHRUTILIPI would be a direction strongly aligned with Prof. Rao's research in Indian language speech processing.

Chapter 11

Conclusion

This report presented a Prototype Speech Recognizer developed across four progressive phases as part of the Advanced Topics in Speech Processing course at IIT Kharagpur.

Phase 1 demonstrated that classical MFCC features combined with scikit-learn classifiers — particularly SVM (RBF kernel) and XGBoost — achieve strong accuracy on a 35-word recognition task with over 105,000 real audio samples. The CNN further improved upon these classical models by treating the 2-D MFCC feature map as an image and learning hierarchical spatial representations. A live microphone demo confirmed that the trained model generalizes correctly to real spoken input.

Phase 2 showed that the same MFCC-plus-classifier paradigm scales to sentence-level recognition on a 100-class command vocabulary, with SVM achieving 88–95% accuracy on TTS-generated speech. The augmentation strategy provided meaningful acoustic diversity within a synthetic corpus, but informal testing on natural speech highlighted the practical consequences of domain mismatch when using TTS-generated training data.

Phase 3 addressed the limitations of both fixed-length feature aggregation and synthetic training data by building a Conv1D + Bidirectional LSTM model trained end-to-end with CTC loss on 100 hours of real LibriSpeech speech. The model achieved a WER of 0.77 on the *test-clean* evaluation set after 10 epochs of training, with the Whisper `base` model achieving 0.31 WER on the same data for comparison. This gap illustrates the practical importance of pre-training at scale.

Phase 4 confirmed that large pre-trained transformer models represent the current practical state of the art for open-vocabulary speech recognition. The Whisper web application achieved near-zero WER on clean English speech without any task-specific fine-tuning, while also providing a rich educational interface for demonstrating spectrogram visualization, noise robustness, model comparison, and word-level error analysis.

Taken together, the four phases form a coherent empirical journey through the field of ASR: from handcrafted cepstral features and classical classifiers, through convolutional deep learning on real word-level data, to end-to-end sequence modeling with CTC, and finally to pre-trained transformer-based transcription. Each step yielded direct experimental evidence of the trade-offs between architectural complexity, data requirements, and recognition performance.

Bibliography

[1] K. S. Rao, "Advanced Topics in Speech Processing," Course Notes, Dept. of CSE, IIT Kharagpur, 2025–2026.

[2] S. Furui, "Speaker-independent isolated word recognition using dynamic features of speech spectrum," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 34, no. 1, pp. 52–59, 1986.

[3] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[4] G. E. Hinton et al., "Deep neural networks for acoustic modeling in speech recognition," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.

[5] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006, pp. 369–376.

[6] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[7] W. Chan, N. Jaitly, Q. Le, and O. Vinyals, "Listen, attend and spell," in *Proc. ICASSP*, 2016, pp. 4960–4964.

[8] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.

[9] H. Ko, V. Peddinti, D. Povey, and S. Khudanpur, "Audio augmentation for speech recognition," in *Proc. INTERSPEECH*, 2015, pp. 3586–3589.

[10] D. S. Park et al., "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. INTERSPEECH*, 2019, pp. 2613–2617.

[11] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[12] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR corpus based on public domain audio books," in *Proc. ICASSP*, 2015, pp. 5206–5210.

[13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[14] V. Vapnik, *The Nature of Statistical Learning Theory*. New York: Springer, 1995.

[15] B. McFee et al., "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conference*, 2015, pp. 18–25.

[16] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.